

东集RFID Android接口调用准备工作

介绍

本章提供有关导入和运行 RFID移动应用程序代码和指令的说明。

安装Android Studio

要安装Android Studio，请访问<https://developer.android.com/studio/>并单击下载ANDROID STUDIO。
该项目使用以下配置：

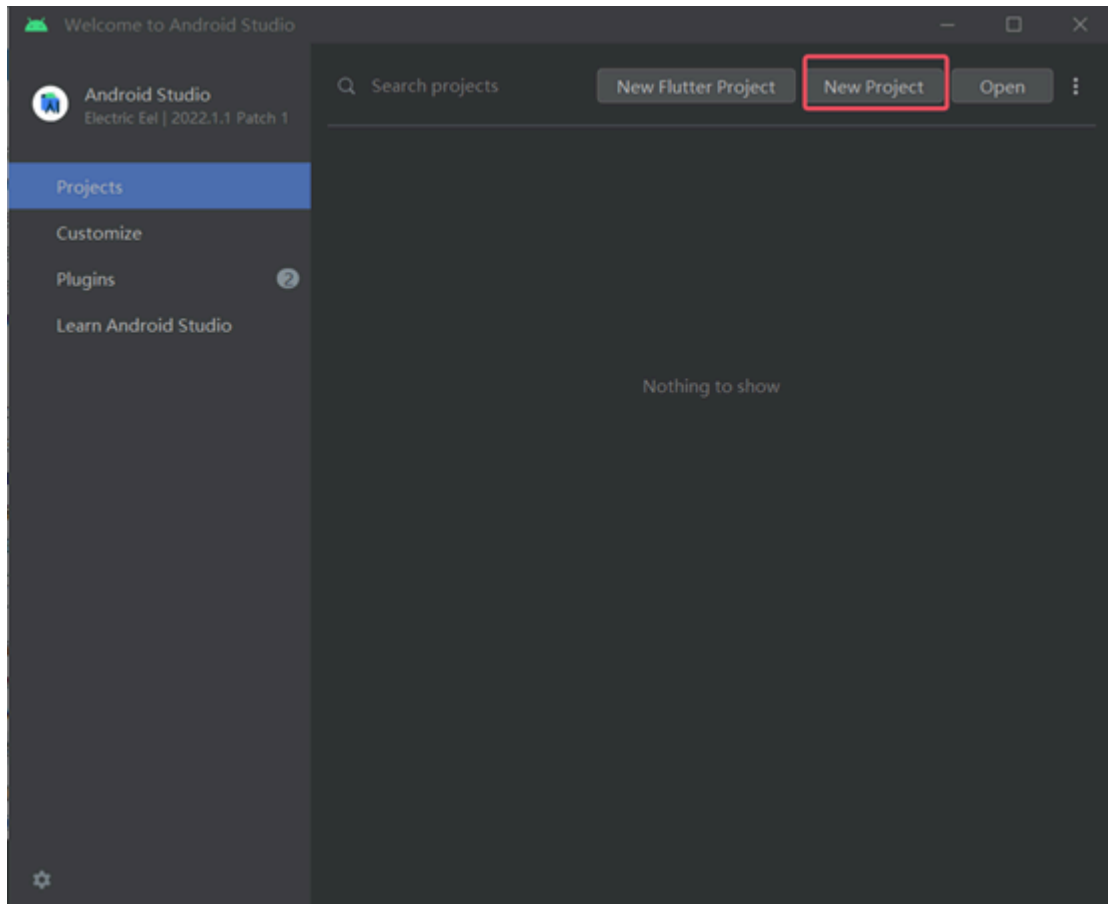
- 1、最低SDK版本- API 19：Android 5.1 (Lollipop)
- 2、目标SDK版本- API 30：Android 11 (Red Velvet Cake)
- 3、Gradle版本- 6.5或与Android Studio版本匹配的最新版本。
- 4、Android Gradle 插件版本4.1.3或与Android Studio版本匹配的最新版本，定义在顶层 build.gradle 中。
- 5、Java 版本：1.8 (Java 8)
- 6、Android Studio 2.3及更高版本。

注：Android Studio附带的最新SDK工具是可以接受的。要使用SDK管理器下载所需的Android SDK包进入菜单工具> Android > SDK管理器

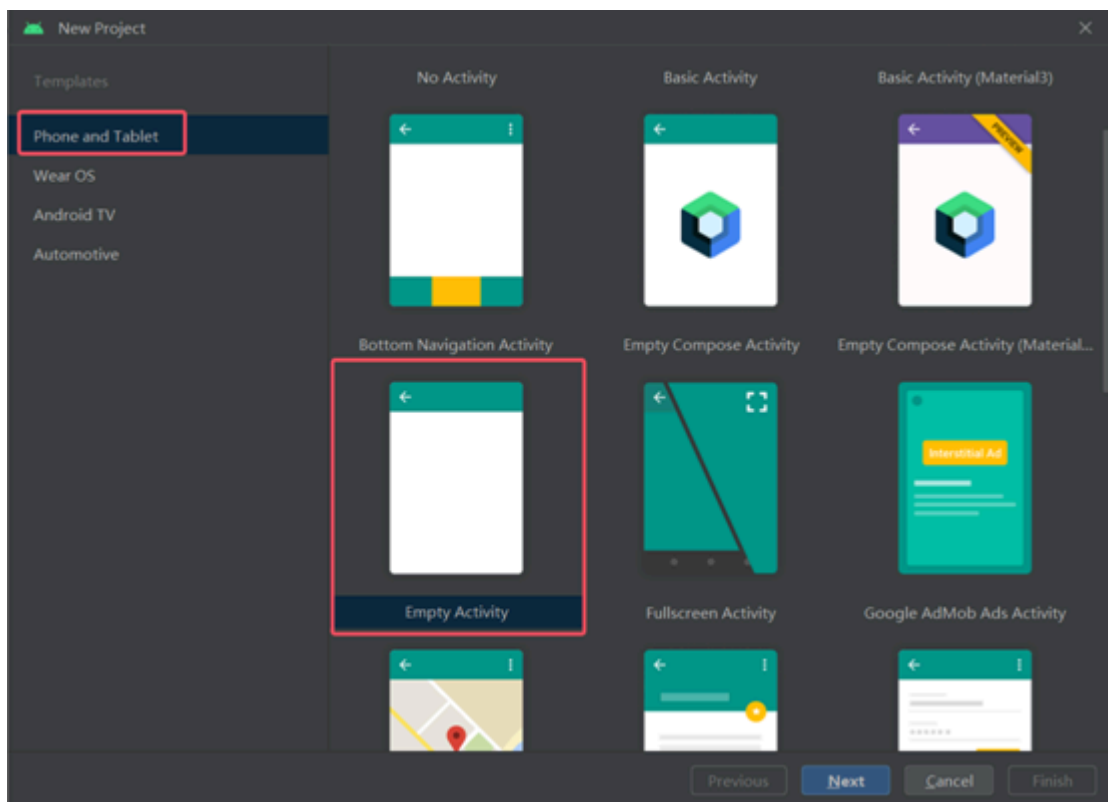
创建项目

首次安装、打开Android Studio可能回先开始下载所需的Gradle/SDK包。定义代理信息
Android Studio和gradl属性，然后Android Studio开始构建项目。

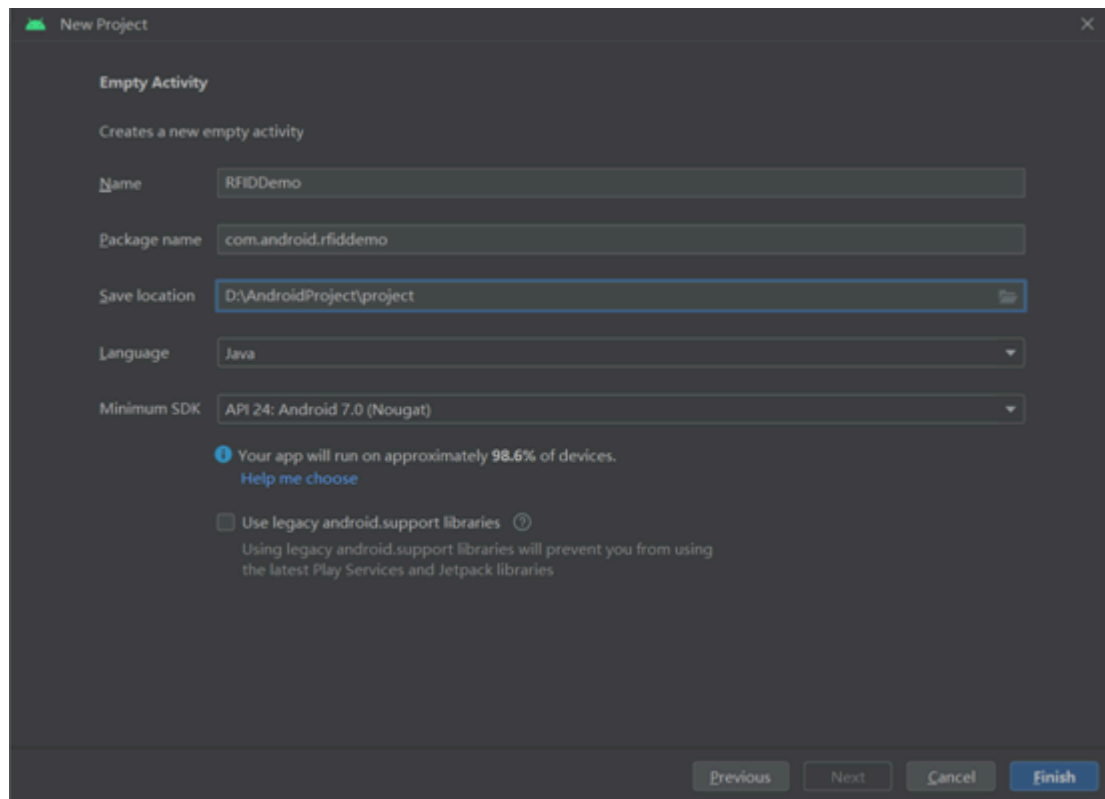
- 1、点击New-Project 进入创建项目界面。



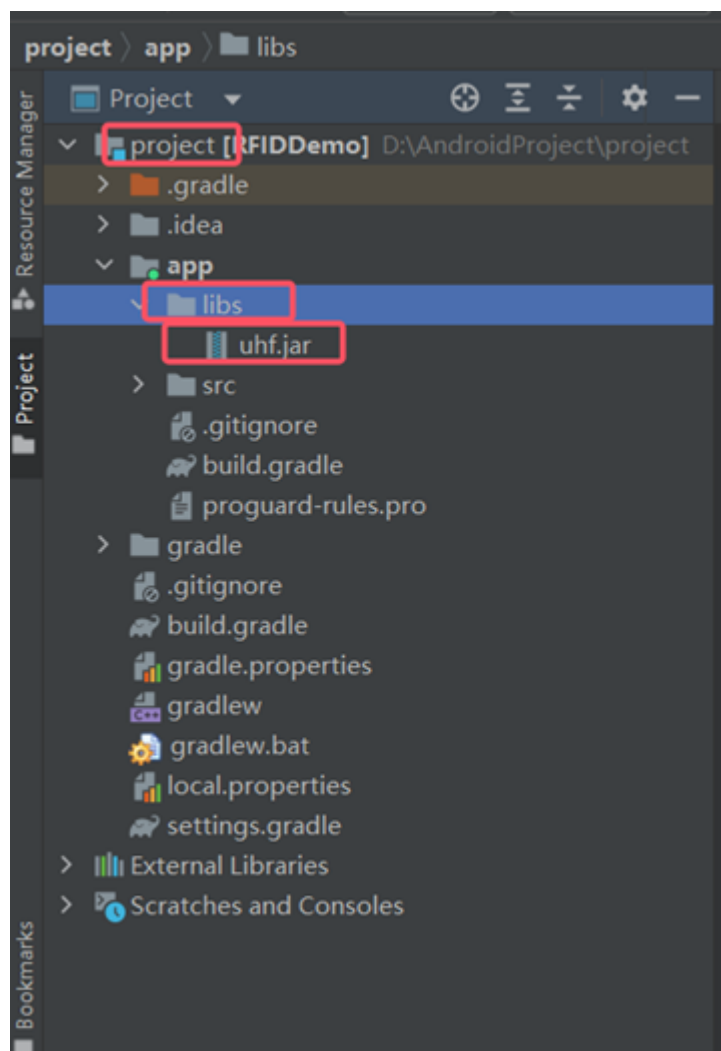
2、选择Phone and Tablet，在右边选择Empty Activity，点击 Next。



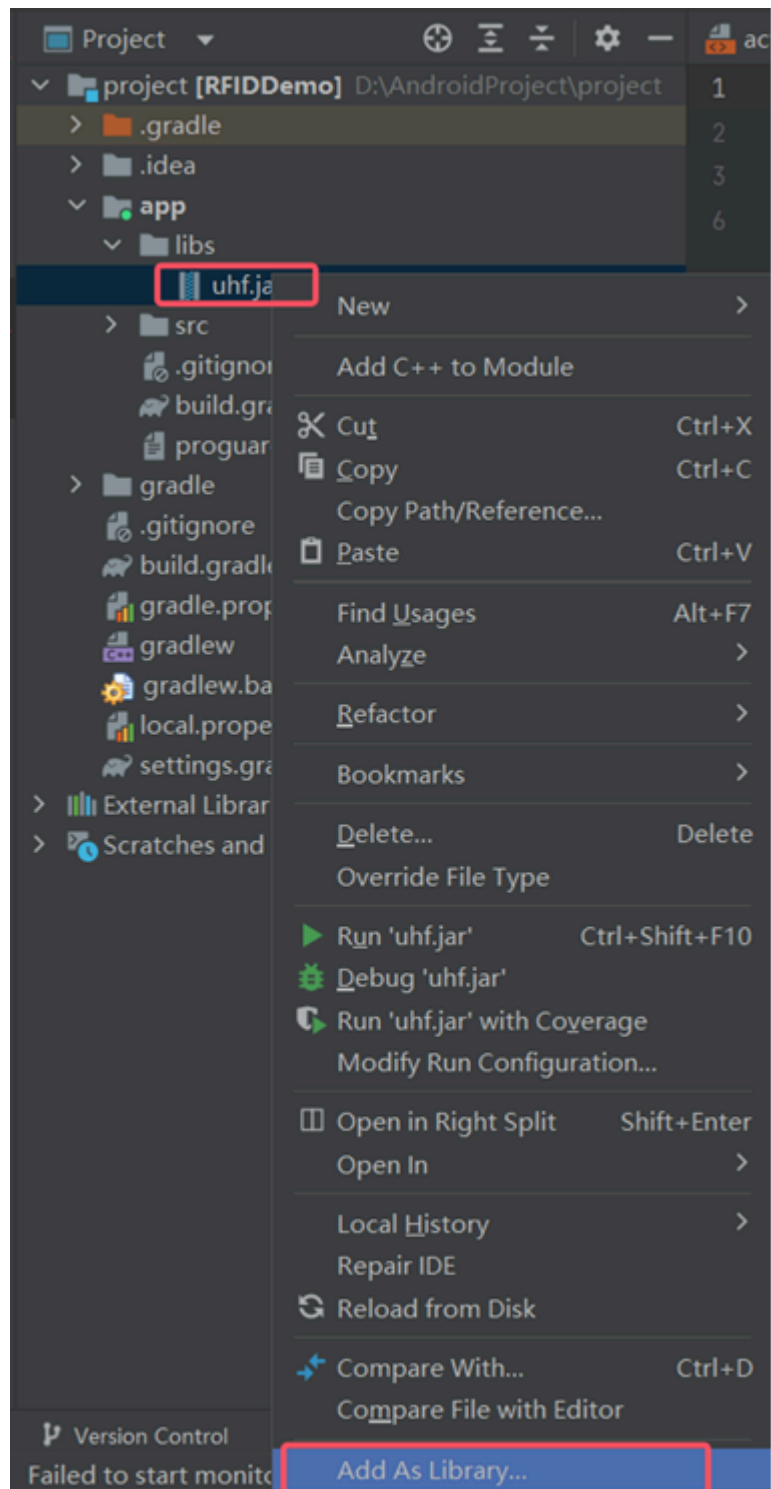
3、设置程序包名、路径、SDK版本等，点击finish。

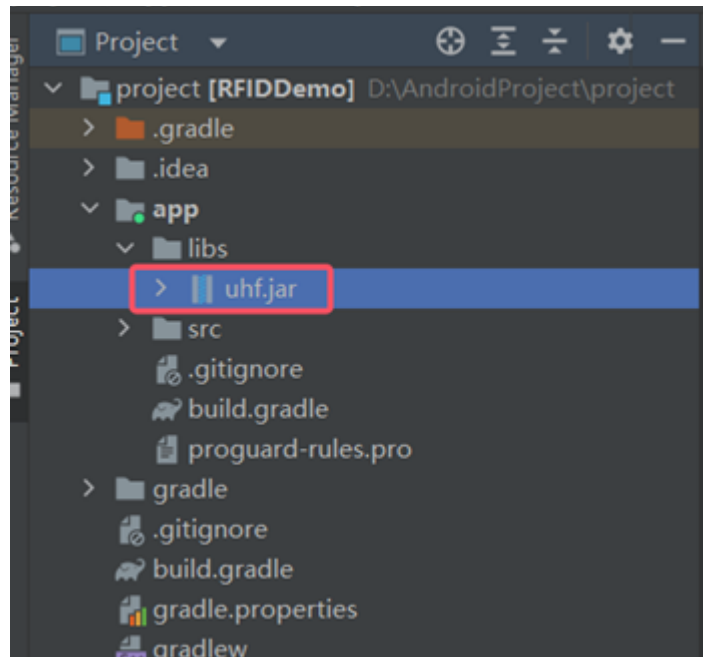


4、我们提供的SDK是jar包的形式，将uhf.jar粘贴到project--app--libs下。



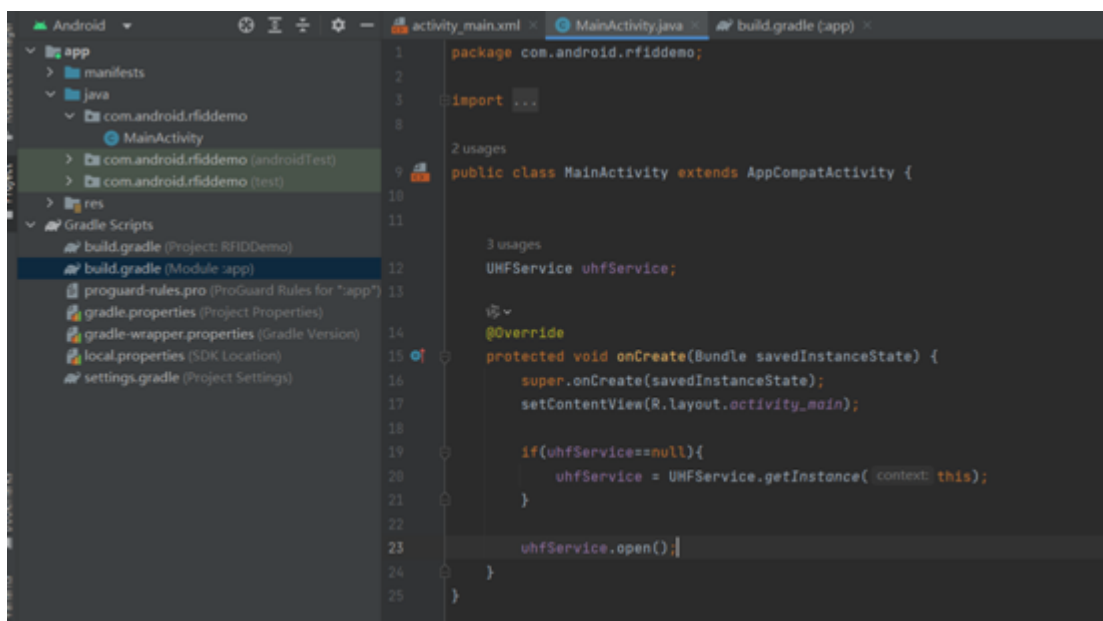
5、右击uhf.jar，选择Add As Library.. 将jar引入项目中





```
implementation 'androidx.appcompat:appcompat:1.7.1'
implementation 'com.google.android.material:material:1.12.0'
implementation 'androidx.constraintlayout:constraintlayout:2.2.1'
implementation files('libs\\uhf.jar')
testImplementation 'junit:junit:4.13.2'
androidTestImplementation 'androidx.test.ext:junit:1.2.1'
androidTestImplementation 'androidx.test.espresso:espresso-core:3.6.1'
```

6、Jar包导入之后，就可以正常调用RFID的相关接口



东集RFID Android接口说明

介绍

本文详细介绍了如何使用东集RFID SDK开发Android应用程序的各种基本和高级功能。

东集RFID Android SDK允许应用程序与连接到移动设备的RFID读写器进行通信。

东集 RFID SDK提供的API可用于外部应用程序管理RFID读写器的连接，并控制连接的RFID读写器。

基础知识

注意：详细的API文档可以在与东集 RFID SDK一起发布的Java类参考指南中找到。

东集 RFID SDK for Android提供了管理RFID读写器连接的能力，对已连接的RFID读写器执行各种操作，配置已连接的RFID读写器，以及了解与已连接的RFID读写器相关的其他信息。

东集 RFID Android SDK由一个“jar”格式的静态Android库组成，该库应该与外部Android应用程序链接。配置Android应用项目以启用东集 RFID Android SDK的说明，请参见连接管理章节导入jar包。

所有可用的api都在com.seuic.uhf下定义。api包中UHFSERVICE是SDK中的根Java类。应用程序使用UHFSERVICE对象的单个实例与特定的读写器进行交互。

使用可用的读写器和UHFSERVICE对象来注册事件，连接读写器，连接成功后，执行所需的操作，如盘存、访问标签。

如果API调用失败，则API会报错，可以通过抓logcat日志来分析详细的报错原因。

连接管理

导入jar包

以Android Studio为例：

在应用工程中，将uhf.jar拷贝到libs文件夹下，在build.gradle中引用uhf.jar

```
dependencies {  
    compileOnly files('libs/uhf.jar')  
}
```

打开读写器

打开是与 RFID 读写器对话的第一步。导入包是使用任何API的第一步。导入包如下：

```
import com.seuic.uhf.UHFSERVICE;
```

创建UHFSERVICE类的实例，并打开读写器

```
mService = UHFService.getInstance(this);  
mService.open();
```

关闭读写器

当应用程序完成对RFID阅读器的连接和操作时，它调用以下API来关闭连接，并释放和清理资源。

```
mService.close();
```

配置读写器

设置功率

不同的读写器芯片支持的功率范围不同，U6/U7/U8/U9支持功率范围1~33dBm，U4/U5支持功率范围1~30dBm。

```
//设置功率30 dBm  
mService.setPower(30);
```

获取功率

```
mService.getPower();
```

设置链路模式

不同的读写器芯片支持的链路模式不同，具体参考下面的表格

```
//设置链路模式(0:P0, 1:P1, 2:P2, 3:P3 ... 25:P25)  
mService.setParameters(UHFService.PARAMETER_LINK_PROFILE, 1);
```

链路模式列表：

Profile	U8 (R2000)	U4/U6 (国产)	U9 (E710 升级支持Gen2X)	U5(E510升级支持Gen2X)	U7(国产)	适用协议
P0	40k FM0 Tari 25us	160k M8 Tari 12.5us	160k M8 Tari 20us/185	160k M8 Tari 20us/185	300k M8 Tari 25us	Gen2
P1	250k M4 Tari 25us	160k FM0 Tari 12.5us	640k M4 Tari 6.25us/148	640k M4 Tari 6.25us/148	300k M4 Tari 25us	Gen2
P2	300k M4 Tari 25us	250k FM0 Tari 12.5us	640k FM0 Tari 6.25us/103	/	640k FM0 Tari 6.25us	Gen2
P3	400k FM0 Tari 6.25us	320k M4 Tari 6.25us	250k M4 Tari 20us/146	250k M4 Tari 20us/146	250k M4 Tari 25us	Gen2
P4	/	/	640/M2/pie: 1.5/ tari_us: 7.5/120	640/M2/pie: 1.5/ tari_us: 7.5/120	/	Gen2
P5	/	/	160/M8/pie: 2.0/ tari_us: 20/382	160/M8/pie: 2.0/ tari_us: 20/382	/	Gen2
P6	/	/	250/M4/pie: 2.0/ tari_us: 20/343	250/M4/pie: 2.0/ tari_us: 20/343	/	Gen2
P7	/	/	640/M4/pie: 1.5/ tari_us: 7.5/345	640/M4/pie: 1.5/ tari_us: 7.5/345	/	Gen2
P8	/	/	640/M2/pie: 2.0/ tari_us: 7.5/323	640/M2/pie: 2.0/ tari_us: 7.5/323	/	Gen2
P9	/	/	640/M1/pie: 2.0/ tari_us: 7.5/302	/	/	Gen2
P10	/	/	160/M8/pie: 2.0/ tari_us: 20/4185	160/M8/pie: 2.0/ tari_us: 20/4185	/	Gen2X
P11	/	/	250/M4/pie: 2.0/ tari_us: 20/4146	250/M4/pie: 2.0/ tari_us: 20/4146	/	Gen2X
P12	/	/	640/M4/pie: 1.5/ tari_us: 7.5/4148	640/M4/pie: 1.5/ tari_us: 7.5/4148	/	Gen2X
P13	/	/	640/M2/pie: 2.0/ tari_us: 7.5/4124	640/M2/pie: 2.0/ tari_us: 7.5/4124	/	Gen2X
P14	/	/	[103, 148, 146, 185]/1120	/	/	Gen2
P15	/	/	[120, 148, 146, 185]/1122	[120, 148, 146, 185]/1122	/	Gen2
P16	/	/	[302, 345, 343, 382]/1320	/	/	Gen2
P17	/	/	[323, 345, 343, 382]/1322	[323, 345, 343, 382]/1322	/	Gen2
P18	/	/	[4185, 185]/5185	[4185, 185]/5185	/	Gen2+Gen2X
P19	/	/	[4146, 146]/5146	[4146, 146]/5146	/	Gen2+Gen2X
P20	/	/	[4148, 148]/5148	[4148, 148]/5148	/	Gen2+Gen2X
P21	/	/	[4124, 124]/5124	[4124, 124]/5124	/	Gen2+Gen2X
P22	/	/	[4382, 382]/5382	[4382, 382]/5382	/	Gen2+Gen2X
P23	/	/	[4343, 343]/5343	[4343, 343]/5343	/	Gen2+Gen2X
P24	/	/	[4345, 345]/5345	[4345, 345]/5345	/	Gen2+Gen2X
P25	/	/	[4323, 323]/5323	[4323, 323]/5323	/	Gen2+Gen2X
P26	/	/	/	/	/	Gen2
P27	/	/	Auto Profile	Auto Profile	/	Gen2X
P28	/	/	160/M8/pie: 2.0/ tari_us: 20/4185+CC1/2	160/M8/pie: 2.0/ tari_us: 20/4185+CC1/2	/	Gen2X
P29	/	/	250/M4/pie: 2.0/ tari_us: 20/4146+CC1/2	250/M4/pie: 2.0/ tari_us: 20/4146+CC1/2	/	Gen2X
P30	/	/	640/M4/pie: 1.5/ tari_us: 7.5/4148+CC1/2	640/M4/pie: 1.5/ tari_us: 7.5/4148+CC1/2	/	Gen2X
P31	/	/	640/M2/pie: 2.0/ tari_us: 7.5/4124+CC1/2	640/M2/pie: 2.0/ tari_us: 7.5/4124+CC1/2	/	Gen2X
P32	/	/	160/M8/pie: 2.0/ tari_us: 20/4382+CC1/2	160/M8/pie: 2.0/ tari_us: 20/4382+CC1/2	/	Gen2X
P33	/	/	250/M4/pie: 2.0/ tari_us: 20/4343+CC1/2	250/M4/pie: 2.0/ tari_us: 20/4343+CC1/2	/	Gen2X
P34	/	/	640/M4/pie: 1.5/ tari_us: 7.5/4345+CC1/2	640/M4/pie: 1.5/ tari_us: 7.5/4345+CC1/2	/	Gen2X
P35	/	/	640/M2/pie: 2.0/ tari_us: 7.5/4323+CC1/2	640/M2/pie: 2.0/ tari_us: 7.5/4323+CC1/2	/	Gen2X
P36	/	/	Auto Profile	Auto Profile	/	Gen2X

说明：此表格第一列列举了所有的profile值，第二列到第六列解释了不同的RFID 芯片下profile的具体值，最后一列说明了适用的协议。

注意：想让profile值设置生效，必须调用setParameters方法设置PARAMETER_S2_ALGORITHM为启用。

设置session和target

```
//session(0:S0, 1:S1, 2:S2, 3:S3)
mService.setParameters(UHFService.PARAMETER_INVENTORY_SESSION, 1);

//target(0:A, 1:B, 2:A/B)
mService.setParameters(UHFService.PARAMETER_INVENTORY_SESSION_TARGET, 0)
```


设置地区频段

不同的读写器芯片支持的地区不同，具体参考下面的表格

```
mService.setRegion("FCC");
```

频段列表：

全球区域RFID频段划分及支持								
region: 区域值	区域频段MHz	U9(E710)		U5(E510)		U8(R2000)		U4/U6/U7 (国产)
		US机型 (EU(Upper Band)/CH机型	EU(Lower Band)机型	US机型 (EU(Upper Band)/CH机型	EU(Lower Band)机型	US机型 (EU(Upper Band)/CH机型	EU(Lower Band)机型	CH机型
China1	China1 (920~925)	支持		支持		支持		支持
FCC	FCC (902~928)	支持		支持		支持		支持
JAPAN	Japan (916.7~920.9)	支持		支持				
MALAYSIA	Malaysia (919~923)	支持		支持				
ETSI	ETSI(865~868)		支持		支持		支持	
VIETNAM	Vietnam (920~923)	支持		支持				
AUSTRALIA	Australia (920~926)	支持		支持				
AUSTRIA	Austria (916.1~918.9)	支持		支持				
BRAZIL	Brazil (915~928)	支持		支持				
ESTONIA	Estonia (915~921)	支持		支持				
HK	HK (920~925)	支持		支持		支持		支持
INDIA	India (865~867)		支持		支持			
INDONESIA	Indonesia (923~925)	支持		支持				
ISRAEL	Israel (915~917)	支持		支持				
KOREA	Korea (917~920.8)	支持		支持				
NEW_ZEALAND	New Zealand (920~928)	支持		支持				
PERU	Peru (915~928)	支持		支持				
PHILIPPINES	Philippines (918~920)	支持		支持				
RUSSIA	Russia (915~928)	支持		支持				
SINGAPORE	Singapore (920~925)	支持		支持				
SOUTH_AFRICA	south africa (915.4~919)	支持		支持				
TAIWAN	Taiwan (922~928)	支持		支持				
THAILAND	Thailand (920~925)	支持		支持				
URUGUAY	uruguay (902~928)	支持		支持				

获取地区频段

```
mService.getRegion();
```

设置定频

读写器设置定频掉电不保存。
如果需要设回跳频调用接口setRegion。

```
//设置固定频点915.250 MHz  
mService.setParameters(UHFService.PARAMETER_FREQUENCY, 915250);
```

设置隐藏PC

隐藏PC号： 盘存只返回标签的EPC

```
// 盘存不带PC号
mService.setParameters(UHFService.PARAMETER_HIDE_PC, 1);

// 盘存带PC号
mService.setParameters(UHFService.PARAMETER_HIDE_PC, 0);
```

其他参数定义

setParameters可以设置多种参数，具体参数定义如下表格：

id (功能参数名)	U8(R2000)	U4/U6(国产)	U7(国产)	U9 (E710)	U5 (E510)	备注
	value值	value值	value值	value值	value值	
PARAMETER_LINK_PROFILE (0)	请参考链路模式列表	请参考链路模式列表	请参考链路模式列表	请参考链路模式列表	请参考链路模式列表	Profile
PARAMETER_INVENTORY_SESSION (1)	0,1,2,3(S0,S1,S2,S3)	0,1,2,3(S0,S1,S2,S3)	0,1,2,3(S0,S1,S2,S3)	0,1,2,3(S0,S1,S2,S3)	0,1,2,3(S0,S1,S2,S3)	Session
PARAMETER_INVENTORY_SESSION_TARGET (2)	0,1(A,B)	0,1(A,B)	0,1,2(A,B,AB)	0,1,2(A,B,AB)	0,1,2(A,B,AB)	Target
PARAMETER_INVENTORY_SPEED (3)	0,1(fast,slow)	/	/	/	/	寻卡速度
PARAMETER_EXTENSIONS_FASTID (5)	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	Fastid
PARAMETER_CLEAR_EPCLIST_WHEN_START_INVENTORY (6)	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	开始寻卡是否清空之前EPC
PARAMETER_HIDE_PC (7)	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	隐藏PC号
PARAMETER_CLEAR_EPCLIST (8)	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	清空EPC
PARAMETER_ALGORITHM_STARTQVALUE (9)	0~15(4)	0~15(4)	0~15(4)	0~15(4)	0~15(4)	初始Q值
PARAMETER_ALGORITHM_MINQVALUE (10)	0~15(0)	/	/	/	/	最小Q值
PARAMETER_ALGORITHM_MAXQVALUE (11)	0~15(15)	/	/	/	/	最大Q值
PARAMETER_ALGORITHM_TOGGLETARGET (13)	0,1(DISABLE,ENABLE)	/	/	/	/	ToggleTarget
PARAMETER_EXTENSIONS_TAGFOCUS (15)	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	TagFocus
PARAMETER_S2_ALGORITHM (34)	/	/	/	0,1(DISABLE,ENABLE)	0,1(DISABLE,ENABLE)	启用情况下，target、session、profile参数是可设置的，设置成多少就是多少 不启用，target、session、profile参数是自适应场景的，设置不生效。
PARAMETER_TAG_EMBEDDED_DATA (30)	EmbeddedData	EmbeddedData	EmbeddedData	EmbeddedData	EmbeddedData	附加数据
PARAMETER_TAG_FILTER (31)	TagFilter	TagFilter	TagFilter	TagFilter	TagFilter	过滤条件

表格说明：此表格第一列列举了setParameters函数所有的参数，第二列到第六列解释了不同的RFID 芯片可以设置的值，最后一列说明了各个参数具体的含义

基本操作

涉及标签操作需要导入标签数据信息类

```
import com.seuic.uhf.EPC;
```

标签信息包括如下:

```
public class EPC {
    public byte[] id;
    public int len;
    public int rssi;
    public int count;
    //获取字符串epcID
    public String getId() {
        throw new RuntimeException("Stub!");
    }
    //获取字符串附加数据
    public String getEmbedded() {
        throw new RuntimeException("Stub!");
    }
}
```

单次盘存标签

单次盘存只返回一张标签数据

```
EPC epc = new EPC();
mService.inventoryOnce(epc, 0);
```

回调函数

连续盘存通过回调函数持续返回标签数据

在开始盘存之前注册回调函数，程序结束时注销回调。

接受回调数据时，函数tagsRead里面不要做耗时操作。

```
//回调函数
private IReadTagsListener mIReadTagsListener = new IReadTagsListener() {
    @Override
    public void tagsRead(List<EPC> epclist) {
        //处理返回到epclist
    }
};

//注册回调
mService.registerReadTags(mIReadTagsListener);

//注销回调
mService.unregisterReadTags(mIReadTagsListener);
```

开始连续盘存

```
mService.inventoryStart();
```

停止连续盘存

```
mService.inventoryStop();
```

读标签

应用程序调用方法mService.readTagData从指定EPC标签的内存读取数据。

```
String epcID = "1234ABCD00000000000001122";//epc
String psw = "00000000";//访问密码
int bank = 3;//0:Reserved, 1:EPC, 2:TID, 3:User
int offset = 0;//从第0个字节开始读
int len = 2;//读取2个字节数据
byte[] data = new byte[2];//定义用于保存读取数据
mService.readTagData(getHexByteArray(epcID),
                    getHexByteArray(psw),
                    bank,
                    offset,
                    len,
                    data);
```

将字符串转换成十六进制形式的数组

```
public static byte[] getHexByteArray(String hexString) {
    if (hexString == null || hexString.equals("")) {
        return null;
    }
    hexString = hexString.toUpperCase();
    hexString = hexString.replace(" ", "");
    byte[] buffer = new byte[hexString.length() / 2];
    int length = hexString.length() / 2;
    char[] hexChars = hexString.toCharArray();
    for (int i = 0; i < length; i++) {
        int pos = i * 2;
        buffer[i] = (byte) (charToByte(hexChars[pos]) << 4 | charToByte(hexChars[pos + 1]));
    }
    return buffer;
}
```

写标签

应用程序调用方法mService.writeTagData向指定EPC标签内存写入数据。

```
String epcID = "1234ABCD00000000000001122";//epc
String psw = "00000000";//访问密码
int bank = 3;//0:Reserved, 1:EPC, 2:TID, 3:User
int offset = 0;//从第0个字节开始写
int len = 2;//写入2个字节数据
byte[] data = new byte[2];//用于保存写入数据
mService.readTagData(getHexByteArray(epcID),
                    getHexByteArray(psw),
                    bank,
                    offset,
                    len,
                    data);
```

锁标签

应用程序调用方法mService.lockTag对指定EPC标签的内存执行锁定操作。

```
String epcID = "1234ABCD00000000000001122";//epc
String psw = "00000000";//访问密码
//0,1,2,3解锁标签 (0: Reserved 1: EPC 2: TID 3: User)
//10,11,12,13锁标签 (10: Reserved 11: EPC 12: TID 13: User)
//20,21,22,23永久锁标签 (20: Reserved 21: EPC 22: TID 23: User)
//30,31,32,33永久解锁标签 (30: Reserved 31: EPC 32: TID 33: User)
int mLockFlag = 13;
mService.lockTag(getHexByteArray(epcID),
                getHexByteArray(psw),
                mLockFlag);
```

销毁标签

应用程序调用mService.killTag方法来销毁一个标签，销毁后的标签无法恢复。

```
String epcID = "1234ABCD00000000000001122";//epc
String psw = "12345678";//销毁密码必须先设置为非0密码
mService.killTag(getHexByteArray(epcID),
                getHexByteArray(psw));
```

高级操作

过滤盘存标签

设置过滤条件，让盘存只返回符合条件的标签数据

```
byte[] val = new byte[255];
val[0] = 1;//1:EPC, 2:TID, 3:User
val[1] = 0;//从第0个字节开始匹配
val[2] = 4;//匹配4个字节数据
val[3] = 0;//0:匹配, 1: 不匹配
String filterdata = "11223344";
byte[] data = getHexByteArray(filterdata);
System.arraycopy(data, 0, val, 4, val[2]);
mService.setParamBytes(UHFService.PARAMETER_TAG_FILTER, val);
```

盘存标签时附加数据

盘存标签时会获取到标签EPC数据，启用附加数据功能后，可以同时读标签其他存储区的数据，如Reserved、TID或者User，一次只能读一个存储区的数据。

```
byte[] embd = new byte[255];
embd[0] = 2;//0:Reserved, 2:TID, 3:User
embd[1] = 0;//从第0个字节开始
embd[2] = 12;//附加12个字节数据
String psw = "00000000";//访问密码
System.arraycopy(getHexByteArray(psw), 0, embd, 3, 4);
mService.setParamBytes(UHFService.PARAMETER_TAG_EMBEDDEDATA, embd);
```

温度标签和LED标签

读温度标签

每个温度标签的算法都是不同的，因此读温度标签之前，需要明确标签的型号，目前已经实现的温度标签型号如下：

型号	TID	参数值
B5020M1-T1E	E201E2	1
LTU32	E2035101	2
NMV2D	E2C19CB1	3
ET-C2ES	E201E2	4
ET-C2ESL	FFFFFF	5
M3D	E28240	6
HX03	E200FF	8

```
String epcID = "1234ABCD00000000000001122";//epc
String psw = "00000000";//访问密码
int Manufacturer = 1;//上图中型号B5020M1-T1E对应的参数值
mService.readTagTemperature(getHexByteArray(epcID),
                             getHexByteArray(psw),
                             Manufacturer);
```

LED标签

每个LED标签的点亮算法都是不同的，因此读温度标签之前，需要明确标签的型号，目前已经实现的温度标签型号如下：

型号	TID	点亮条件
KX2005X-BL	E281D0	Reserved区、偏移8字节、读2字节
N2ESL	E201E2	User区、偏移224字节、读2字节
VBL	E2C19C	Reserved区、偏移10字节、读2字节

点亮KX2005X-BL型号标签示例如下：

```
//设置附加数据
byte[] embd = new byte[255];
embd[0] = 0;
embd[1] = 8;
embd[2] = 2;
String psw = "00000000";
System.arraycopy(getHexByteArray(psw), 0, embd, 3, 4);
mService.setParamBytes(UHFService.PARAMETER_TAG_EMBEDDEDATA, embd);
//开始盘存
mService.inventoryStart();
```

LED标签常亮（只有U9支持）

LED标签持续亮灯，不会闪烁。
常亮状态会消耗更多的功耗，RFID模块温度也会更高，建议不要让标签长时间常亮。
实现KX2005X-BL型号标签常亮示例如下：

```
//每次模块上电打开后都需要设置一下常亮模式
byte[] startCMD = new byte[1];
byte[] resCMD = new byte[1];
startCMD[0] = 0x08;
boolean ret = mService.IOControl(47,startCMD,1,resCMD,1,100);
//设置固定频点915.250MHz
mService.setParameters(UHFService.PARAMETER_FREQUENCY, 915250);
//设置附加数据
byte[] embd = new byte[255];
embd[0] = 0;
embd[1] = 8;
embd[2] = 2;
```

```
String psw = "00000000";
System.arraycopy(getHexByteArray(psw), 0, embd, 3, 4);
mService.setParamBytes(UHFService.PARAMETER_TAG_EMBEDDEDATA, embd);
//开始盘存
mService.inventoryStart();
```

扩展接口

读标签扩展

应用程序调用方法mService.readTagDataEx可以指定EPC、TID或者User区读取数据。

```
//指定过滤TID
String TID = "E280110C20007582C3480A11";//TID
int filterbank = 2;//1:EPC, 2:TID, 3:User
int filteroffset = 0;//从第0个字节开始匹配
int filterlen = 12;//匹配12个字节数据
//读标签的参数
String psw = "00000000";//访问密码
int bank = 3;//0:Reserved, 1:EPC, 2:TID, 3:User
int offset = 0;//从第0个字节开始读
int len = 2;//读取2个字节数据
byte[] data = new byte[2];//定义用于保存读取数据
mService.readTagData(getHexByteArray(TID),
                    filterbank,
                    filteroffset,
                    filterlen,
                    getHexByteArray(psw),
                    bank,
                    offset,
                    len,
                    data);
```

写标签扩展

应用程序调用方法mService.writeTagDataEx可以指定EPC、TID或者User区写入数据。

```
//指定过滤TID
String TID = "E280110C20007582C3480A11";//TID
int filterbank = 2;//1:EPC, 2:TID, 3:User
int filteroffset = 0;//从第0个字节开始匹配
int filterlen = 12;//匹配12个字节数据
//写标签的参数
String psw = "00000000";//访问密码
int bank = 3;//0:Reserved, 1:EPC, 2:TID, 3:User
int offset = 0;//从第0个字节开始写
int len = 2;//写入2个字节数据
byte[] data = new byte[2];//用于保存写入数据
mService.writeTagData(getHexByteArray(TID),
                    filterbank,
```



```
filteroffset,  
filterlen,  
getHexByteArray(psw),  
bank,  
offset,  
len,  
data);
```

锁标签扩展

应用程序调用方法mService.lockTagEx可以指定EPC、TID或者User区锁定标签。

```
//指定过滤TID  
String TID = "E280110C20007582C3480A11";//TID  
int filterbank = 2;//1:EPC, 2:TID, 3:User  
int filteroffset = 0;//从第0个字节开始匹配  
int filterlen = 12;//匹配12个字节数据  
//锁标签的参数  
String psw = "00000000";//访问密码  
//0,1,2,3解锁标签（0: Reserved 1: EPC 2: TID 3: User）  
//10,11,12,13锁标签（10: Reserved 11: EPC 12: TID 13: User）  
//20,21,22,23永久锁标签（20: Reserved 21: EPC 22: TID 23: User）  
//30,31,32,33永久解锁标签（30: Reserved 31: EPC 32: TID 33: User）  
int mLockFlag = 13;  
mService.lockTag(getHexByteArray(TID),  
                filterbank,  
                filteroffset,  
                filterlen,  
                getHexByteArray(psw),  
                mLockFlag);
```

销毁标签扩展

应用程序调用mService.killTagEx可以指定EPC、TID或者User区销毁一个标签，销毁后的标签无法恢复。

```
//指定过滤TID  
String TID = "E280110C20007582C3480A11";//TID  
int filterbank = 2;//1:EPC, 2:TID, 3:User  
int filteroffset = 0;//从第0个字节开始匹配  
int filterlen = 12;//匹配12个字节数据  
//销毁标签的参数  
String psw = "00000000";//销毁密码，必须先设置非0密码  
mService.killTag(getHexByteArray(TID),  
                filterbank,  
                filteroffset,  
                filterlen,  
                getHexByteArray(psw))
```

Gen2X功能（只有U5/U9支持）

TagFocus

TagFocus功能提升大规模标签群组的读取可靠性。

该功能使读写器能够指示标签持续刷新其A/B标志位状态，从而保持非响应状态。通过命令已被盘点的标签在搜寻未盘点的标签期间保持静默，读写器发现难读取标签的成功率将显著提高。

```
//0:关闭， 1: 开启  
mService.setParameters(UHFService.PARAMETER_EXTENSIONS_TAGFOCUS, 1);
```

FastID

FastID功能要求所有标签芯片在盘点过程中将其TID附加到EPC之后。

```
//0:关闭， 1: 开启  
mService.setParameters(UHFService.PARAMETER_EXTENSIONS_FASTID, 1);
```

近距离模式

近距离模式减少了90%的读取范围，以防止在远处读取标签数据。

实现方法如下：

先设置一个非0密码。

用非0密码读密码区，地址偏移8字节，读2字节，读到的数据修改bit[4]=1。

```
C068: 1100 0000 0110 1000  
C078: 1100 0000 0111 1000
```

将修改后的数据再用非0密码写回密码区，地址偏移8字节，写2字节

如果需要关闭此功能，修改bit[4]=0。

```
//开启近距模式  
//先读出password区 offset: 8 len: 2  
String epcID = "1234ABCD00000000000001122";//epc  
String psw = "12345678";//访问密码  
byte[] data = new byte[2];  
mService.readTagData(getHexByteArray(epcID), getHexByteArray(psw), 0, 8, 2, data);  
data[1] = (byte)(data[1] | (1 << 4));  
mService.writeTagData(getHexByteArray(epcID), getHexByteArray(psw), 0, 8, 2, data);  
//关闭近距模式  
data[1] = (byte)(data[1] & ~(1 << 4));  
mService.writeTagData(getHexByteArray(epcID), getHexByteArray(psw), 0, 8, 2, data);
```

保护模式

Impinj保护模式使标签对读写器不可见，并使用安全PIN对读写器可见

实现方法如下：

先设置一个非0密码。

用非0密码读密码区，地址偏移8字节，写2字节，修改bit[1]=1。

```
C068: 1100 0000 0110 1000
C06A: 1100 0000 0110 1010
```

将修改后的数据再用非0密码写回密码区，地址偏移8字节，写2字节。

保护模式开启后，正常盘存无法找到标签，需要通过过滤访问密码盘存，过滤访问密码参数如下：target=0，action=1，bank=3，offset=0，len=4，data=4字节密码。

如果需要关闭此功能，修改bit[1]=0。

```
//开启保护模式
//先读出password区 offset: 8 len: 2
String epcID = "1234ABCD000000000000001122";//epc
String psw = "12345678";//访问密码
byte[] data = new byte[2];
mService.readTagData(getHexByteArray(epcID), getHexByteArray(psw), 0, 8, 2, data);
data[1] = (byte)(data[1] | (1 << 1));
mService.writeTagData(getHexByteArray(epcID), getHexByteArray(psw), 0, 8, 2, data);
//关闭保护模式
//先读出password区 offset: 8 len: 2
data[1] = (byte)(data[1] & ~(1 << 1));
mService.writeTagDataEx(3,0,4,getHexByteArray(psw), getHexByteArray(psw), 0, 8, 2, data);
//启用保护模式后,要先过滤密码区后才能盘存到标签
byte[] val = new byte[255];
val[0] = 0;//target
val[1] = 1;//action
val[2] = 3;//bank user
val[3] = 0;//offset
val[4] = 4;//密码长度
val[5] = 0;//匹配
byte[] data = getHexByteArray(psw);
System.arraycopy(data, 0, val, 6, val[4]);
mService.setParamBytes(27, val);
```

标签身份验证

获取标签加密数据，验证标签的真实性。

实现方法如下：

将6个字节随机数发送给标签，标签回复12字节TID+8字节加密后的数据

6个字节随机数的第一个字节的前6个bit需要是00001：表示返回64bitsTID数据。

返回数据格式：1字节TID长度+TID数据+1个字节加密数据长度+加密数据

```
byte[] sendCMD = new byte[11];
byte[] resCMD = new byte[32];
String random = "051122334455";
sendCMD[0] = 1;//标签型号 1:M775, 2:C899
sendCMD[1] = 1;//send_rep
sendCMD[2] = 1;//inc_rep_len
sendCMD[3] = 1;//csi
sendCMD[4] = 6;//数据长度
System.arraycopy(getHexByteArray(random), 0, sendCMD, 5, 6);//6个字节随机数
mService.IOControl(52, sendCMD, 11, resCMD, 32, 0);
byte[] tid = new byte[12];
byte[] encryptedData = new byte[8];
System.arraycopy(resCMD, 1, tid, 0, resCMD[0]);
System.arraycopy(resCMD, resCMD[0]+1+1, encryptedData, 0, resCMD[resCMD[0]+1]);
```